Aston University
Computational Intelligence

Implementing Local Search

Expected Outcomes:
In this lab, you will be implementing two local search methods for the Travelling Salesman Problem (TSP): one close to that discussed in lectures, and one of your own design.

IMPORTANT: A note on working together. You are welcome to work together with friends on the majority of this lab. However, the work for step 6 (creating a novel local search algorithm) may be used as part of your assessment, so it is important that, for step 6, you create and implement the part of your own method which is different from the algorithm you were asked to implement in steps 1-5.

Instructions:
Recall the four types of component that make up a local search algorithm:

- an initialisation method which provides the starting point for local search,
- a neighbourhood function which builds the set of "local" points which a local search method will consider moving to,
- a step function which determines which neighbour to visit,
- a termination condition which determines when we should stop searching.

We will need to implement at least one component of each type. For our first local search algorithm, we will use:

- random initialisation,
- the 2-city-swap neighbourhood,
- the best neighbour step function,
- a time-based termination condition.

Along the way, we will use the components to build a random search algorithm.

Step 1: Random Initialisation
In the previous lab, you should have implemented a method to generate a random route for a given TSP instance. As we are planning on using random initialisation, all we need is a method to generate random routes, so our job is already done for us!

Step 2: Random Search and CPU-time-based Termination.
Our local search algorithm will have a time-based termination condition. In this step, you will start writing the condition by creating a time-limited random search algorithm.

Write a method to which you can pass a TSP instance and a time limit (in seconds). Until the time limit is reached, the method should generate random TSP tours, storing the best generated so far. When the time limit is reached, the best tour found, and its length, should be returned.

Test your method by running it on the Ulysses16 TSP instance with different time limits. How does the quality of solution found vary with the time limit?

Step 3: Generating Random Routes
One way of generating a neighbour of a given tour would be to swap two cities in the tour. The resulting tour should still be valid and, in expectation, should be similar in length to the original tour.

Warning: If you have assumed a fixed starting city and/or included the loop back to the starting city in your tour representation, make sure not to break this assumption by swapping elements of the tour which you are assuming are fixed.

A neighbourhood is the set of all neighbours of a given solution. Write a function `city_swap_neighbourhood` which can be called as follows:

```
route = #Some TSP route
neighbouring_routes = city_swap_neighbourhood(route)
```

such that `neighbouring_routes` will then contain all city-swap-beased neighbours of the original route. If we assume that we have constrained routes to start and end at city 0 and that we include city 0 at the start and end of our route data structure, then a the result of calling this function on the route `[0,1,2,3,4,0]` should be:

```
[[0,2,1,3,4,0],
 [0,3,2,1,4,0],
 [0,4,2,3,1,0],
 [0,1,3,2,4,0],
 [0,1,4,3,2,0],
 [0,1,2,4,3,0]]
```

Warning: A sensible way to implement this would be, for each neighbour, to copy the original route and then to make the swap on the copied route. Note, however, that python (and many other languages) will make shallow copies of lists if you just use simple assignment. This can cause problems as swaps you make to the copy of the array will also change the original array (and any shallow copies of the array you made previously)! To make a deep copy of an array in python you can use the `copy.deepcopy` method.

Step 4: Best Neighbour Step Function
The best neighbour step function evaluates all tours in a neighbourhood and moves to the shortest one. Write a method which takes a neighbourhood (with the same structure as that returned by your neighbourhood method above) and returns the shortest tour in the neighbourhood.

Step 5: Putting It Together
Now that you have implemented the four components of local search, combine them to form a local search algorithm. The structure of your local search algorithm will be very similar to that of

your random search algorithm. However, after generating a random tour it should repeatedly try to make the best local improving move (i.e. generate the neighbourhood of the tour and move to the best neighbouring tour) until it reaches a local optimum (a tour with no shorter neighbouring tours). When a local optimum is reached, it should generate a new random solution and start the local search procedure from there. This process should continue until the set time limit expires.
Test your algorithm by comparing it to random search given the same time limit. How does the quality of solution of the two algorithms compare? How does it vary with the size of the TSP instance?

Step 6: Create A Novel Variant of a Local Search Algorithm
You have now implemented the algorithm discussed in lectures. Can you improve on it? Keeping the termination condition the same, try and implement a different local search algorithm. You could, for example, try implementing the 2-opt neighbourhood discussed in lectures or a different step function to the best neighbour function discussed above. You may also want to consult some of the literature given on the last slide of the slide pack on local search for inspiration.

Test your own local search algorithm against the one recommended above. Is yours better or worse? How would you decide? How confident are you in your answer??